Kada dobar odgovor nije dovoljan dokaz

AI Bootcamp — Nedelja 9

Prethodno čitanje | Predviđeno vreme: 10–15 minuta

1. Aplikacija radi. Da li je to dovoljno?

Tokom prethodnih nedelja upoznali ste se sa razvojem AI aplikacija, korišćenjem jezičkih modela, RAG arhitekturom i osnovnim bezbednosnim principima.

Sada ćemo se baviti pitanjem koje se u profesionalnom razvoju softvera ne može zaobići:

Kako znamo da AI aplikacija zaista radi ono što očekujemo — i kako to možemo da dokažemo?

Zamislite da ste razvili AI asistenta koji odgovara na pitanja korisnika. Testirate ga sa nekoliko pitanja i dobijate odlične odgovore. Asistent koristi odgovarajuće dokumente, odgovori zvuče uverljivo, a aplikacija ne prijavljuje greške.

Da li je spremna za produkciju?

Ne možemo to da zaključimo samo na osnovu nekoliko uspešnih primera.

Možda asistent daje netačne odgovore kada ne pronađe potrebne podatke. Možda koristi zastarele informacije. Možda ispravno odgovara na jednostavna pitanja, ali greši kada mora da poveže više izvora.

A možda su testovi kojima proveravamo aplikaciju toliko površni da ne primećuju nijedan od tih problema.

Upravo time ćemo se baviti tokom devete nedelje: **kako da proverimo ponašanje AI aplikacije, razumemo uzroke grešaka i procenimo da li su naše izmene zaista donele poboljšanje.**

Nećemo razvijati novu aplikaciju od nule. Radićemo sa unapred pripremljenim sistemom, poznatim scenarijima i kontrolisanim uslovima.

Cilj nije da popravimo što više grešaka, već da naučimo kako se pouzdanost softvera proverava na osnovu konkretnih dokaza.

2. Primer: odgovor koji zvuči ispravno

Zamislite asistenta korisničke podrške koji može da proveri status naloga.

Korisnik pita:

„Kakav je status naloga 12345?”

Asistent odgovara:

„Status naloga 12345 je active.”

Na prvi pogled, sve izgleda u redu.

Odgovor je jasan, kratak i odgovara na postavljeno pitanje.

Automatski test koji proverava da odgovor nije prazan prolazi. Čak bi i AI model zadužen za ocenjivanje kvaliteta odgovora mogao da ga oceni kao dobar, ukoliko proverava samo jasnoću i relevantnost.

Međutim, postoji problem.

U evidenciji ne postoji nalog sa identifikatorom 12345.

Alat kojim asistent proverava naloge vratio je rezultat not_found, ali je asistent ipak saopštio korisniku da je nalog aktivan.

Odgovor je uverljiv, ali netačan.

Šta nam ovaj primer govori?

Prvo, uspešno izvršavanje aplikacije ne znači da je rezultat ispravan.

Drugo, kvalitet odgovora ne možemo procenjivati samo na osnovu njegovog izgleda ili formulacije.

Treće, test koji prolazi nije nužno dobar test.

Da bismo otkrili problem, moramo unapred znati kako aplikacija treba da se ponaša, imati uvid u ono što se zaista dogodilo i koristiti provere koje mogu da prepoznaju odstupanje.

To nas dovodi do nekoliko ključnih pojmova.

3. Evaluacija: šta zapravo proveravamo?

U klasičnom softverskom inženjerstvu često možemo precizno da definišemo očekivani rezultat.

Ako funkcija sabira dva broja, lako proveravamo da li je 2 + 3 = 5.

Kod AI aplikacija situacija je složenija.

Na pitanje „Objasni mi kako funkcioniše RAG” moguće je dati više različitih, a podjednako ispravnih odgovora.

Zato ne možemo uvek jednostavno porediti generisani tekst sa jednom unapred zadatom rečenicom.

Umesto toga, definišemo **šta smatramo prihvatljivim ponašanjem**.

Na primer, za asistenta korisničke podrške možemo postaviti sledeće kriterijume:

- Ako nalog postoji, asistent mora da prikaže status koji odgovara podacima iz evidencije.
- Ako nalog ne postoji, asistent mora jasno da saopšti da nije pronađen.
- Asistent ne sme da izmišlja status naloga.
- Ako alat za proveru nije dostupan, asistent ne sme da predstavi pretpostavku kao potvrđenu informaciju.

Ovakvi uslovi nazivaju se **kriterijumi prihvatanja** (*acceptance criteria*).

Na osnovu njih pripremamo evaluacioni skup (*evaluation dataset*) — kolekciju primera kojima proveravamo ponašanje aplikacije.

Svaki primer može da sadrži ulazne podatke, očekivano ponašanje i način provere rezultata.

Važno je razumeti da evaluacioni skup ne mora da sadrži sve moguće situacije. To najčešće nije ni izvodljivo.

Ako aplikacija prođe svih 20 pripremljenih testova, možemo da tvrdimo da je zadovoljila kriterijume tih testova.

Ne možemo automatski da tvrdimo da će ispravno odgovoriti na svako buduće pitanje.

Ko ocenjuje odgovore?

Neke provere možemo da izvršimo običnim programskim kodom.

Na primer, lako je proveriti da li odgovor sadrži obavezno polje ili da li je status naloga jednak vrednosti koju je vratio alat.

Za složenije odgovore možemo koristiti AI model kao ocenjivača (*LLM-as-a-judge*).

Takav model može da procenjuje relevantnost, potpunost ili usklađenost odgovora sa izvorima.

Međutim, i ocenjivač može da pogreši.

Može da prihvati netačan odgovor ili odbaci odgovor koji je zapravo ispravan.

Zato je važno da razumemo **šta određena evaluacija meri, kako donosi odluku i koja su njena ograničenja**.

Visoka ocena sama po sebi nije dokaz pouzdanosti.

4. Trag izvršavanja: kako saznajemo šta se dogodilo?

Kada aplikacija napravi grešku, nije dovoljno samo konstatovati da je odgovor pogrešan.

Potrebno je razumeti kako je do njega došlo.

Zamislite da AI asistent koristi sledeći postupak:

1. Prima korisničko pitanje.
2. Pretražuje dokumente ili poziva odgovarajući alat.
3. Priprema podatke za jezički model.
4. Generiše odgovor.
5. Vraća odgovor korisniku.

Greška može nastati u bilo kom od ovih koraka.

Možda je pretraga vratila pogrešan dokument.

Možda je alat vratio tačan podatak, ali ga model nije pravilno iskoristio.

Možda je aplikacija pogrešno obradila rezultat alata.

Trag izvršavanja (execution trace) omogućava nam da pregledamo zabeležene korake i ustanovimo šta se tokom izvršavanja dogodilo.

U našem primeru sa nepostojećim nalogom, trag bi mogao da pokaže:

Korisnički zahtev:
  Proveri status naloga 12345

Poziv alata:
  lookup_account("12345")

Rezultat alata:
  not_found

Generisani odgovor:
  Status naloga 12345 je active.

Sada imamo mnogo konkretniju sliku problema.

Znamo da alat nije pronašao nalog, ali je korisniku ipak saopšten status.

I dalje treba istražiti zašto se to dogodilo, ali više ne polazimo samo od pretpostavki.

Trag izvršavanja ne predstavlja zapis skrivenog rezonovanja modela. On sadrži informacije o koracima koje je aplikacija izvršila i koje smo odlučili da beležimo.

Odakle potiču podaci?

Pored samog toka izvršavanja, često je važno da znamo:

- Koja verzija aplikacije je korišćena?
- Koji dokumenti ili podaci su bili dostupni?
- Koja konfiguracija je bila aktivna?
- Koji model je korišćen?
- Kojim testom je rezultat proveravan?

Ove informacije nam omogućavaju da pratimo **izvore podataka i poreklo rezultata (provenance)**.

Trag izvršavanja i informacije o izvorima zajedno nam pomažu da istražimo grešku i proverimo zaključke.

Ali ni detaljan trag ne dokazuje automatski da je odgovor tačan. On pokazuje šta je zabeleženo tokom izvršavanja.

5. Kontrolisani eksperimenti: da li smo zaista nešto poboljšali?

Pretpostavimo da smo pronašli problem i izmenili aplikaciju.

Ponovo pokrećemo evaluaciju.

Pre izmene aplikacija je prolazila 70% testova.

Posle izmene prolazi 90%.

Odlično! Ili možda ipak nije?

Šta ako smo istovremeno promenili:

- Verziju jezičkog modela.
- Dokumente koje aplikacija koristi.
- Konfiguraciju pretrage.
- Sam evaluacioni skup.

Sada ne možemo pouzdano da utvrdimo šta je izazvalo poboljšanje.

Možda je naša izmena pomogla.

Možda je novi model bolji.

Možda su testovi jednostavniji.

Zato u softverskom inženjerstvu koristimo **kontrolisane eksperimente**.

Pokušavamo da zadržimo iste uslove i promenimo samo ono čiji uticaj želimo da ispitamo.

Za smisleno poređenje potrebno je da znamo koje verzije koda, podataka, konfiguracije i evaluacije su korišćene.

Ovo nazivamo **reproduktivnošću** (*reproducibility*) — mogućnošću da ponovimo eksperiment pod dokumentovanim uslovima.

Kod AI sistema potpuno identični rezultati nisu uvek garantovani, čak ni kada koristimo iste parametre.

Ipak, kontrolisani uslovi nam omogućavaju da donosimo znatno pouzdanije zaključke.

Šta ako popravka pokvari nešto drugo?

Zamislite da smo rešili problem sa nepostojećim nalogom.

Sada asistent ispravno odgovara:

„Nalog nije pronađen.”

Ali nakon izmene počinje da odbija i zahteve za naloge koji postoje.

Popravili smo jedan problem, a napravili drugi.

To nazivamo **regresijom** (*regression*).

Zato posle izmene ne proveravamo samo slučaj koji smo popravljali.

Ponovo pokrećemo i prethodno definisane provere kako bismo ustanovili da li je ranije ispravno ponašanje ostalo očuvano.

6. Kada je aplikacija spremna za produkciju?

Zamislite dve aplikacije.

Prva prolazi 95 od 100 testova. Preostalih pet neuspešnih slučajeva odnose se na manje probleme sa formatiranjem odgovora.

Druga prolazi 99 od 100 testova. Jedini neuspešan slučaj predstavlja neovlašćeno otkrivanje poverljivih podataka.

Koja aplikacija je spremnija za produkciju?

Sam procenat uspešnih testova nije dovoljan da odgovorimo.

Moramo razumeti **koje vrste grešaka postoje i kakve posledice mogu da izazovu**.

Neki problemi su prihvatljivi u određenim okolnostima. Drugi mogu biti razlog da se aplikacija ne pusti u rad, bez obzira na ukupnu ocenu.

Zato se odluka o puštanju aplikacije u rad (*release decision*) ne zasniva samo na jednom rezultatu evaluacije.

Potrebno je razmotriti:

- Koji kriterijumi su ispunjeni?
- Koji problemi su i dalje prisutni?
- Koliko su ozbiljne njihove moguće posledice?
- Šta još nismo proverili?
- Kako ćemo prepoznati problem nakon puštanja aplikacije u rad?
- Kada bi trebalo ograničiti funkcionalnost ili vratiti prethodnu verziju?

Važno je razumeti i da se provera pouzdanosti ne završava puštanjem aplikacije u produkciju.

Novi podaci, promene modela, drugačije ponašanje korisnika i izmene sistema mogu dovesti do novih problema.

Zbog toga je praćenje ponašanja aplikacije (*monitoring*) sastavni deo njenog životnog ciklusa.

7. Gde se uklapa Codex?

Tokom ove nedelje koristićete Codex kao pomoć pri istraživanju problema, pisanju koda i pokretanju provera.

Možete ga koristiti da:

- Analizira postojeći kod.
- Predloži uzrok greške.
- Objasni trag izvršavanja.
- Predloži dodatne testove.
- Implementira izmenu.
- Pokrene evaluaciju i pripremi pregled rezultata.

Međutim, postoji važna razlika između **predloga rešenja i potvrde da je rešenje ispravno**.

Zamislite da Codex:

1. Pronađe problem.
2. Napiše izmenu.
3. Napiše test za tu izmenu.
4. Pokrene test i saopšti da je problem rešen.

To može biti sasvim dobar razvojni postupak.

Ali ako test ne proverava pravi kriterijum, njegov uspešan rezultat neće potvrditi ono što želimo da znamo.

Zato je važno da razumete šta je zaista provereno i da li dostupni dokazi podržavaju zaključak.

Cilj nije da radite bez AI asistenta. Cilj je da naučite kako da proveravate rezultate rada u kojem AI aktivno učestvuje.

To je važna veština savremenog softverskog inženjerstva.

8. Šta vas očekuje ove nedelje?

U petak ćemo kroz objašnjenja, demonstracije i diskusiju proći ceo postupak provere pouzdanosti AI aplikacije.

Videćete kako evaluacija može da pruži pogrešan osećaj sigurnosti, kako se pomoću tragova izvršavanja istražuju problemi i zašto su kontrolisani uslovi važni za procenu poboljšanja.

Nakon predavanja, samostalno ćete u parovima raditi na unapred pripremljenoj aplikaciji.

Vaš zadatak neće biti da otklonite sve postojeće greške.

Biće potrebno da istražite jedan problem, sprovedete ciljanu izmenu, proverite rezultate i obrazložite da li dostupni dokazi opravdavaju puštanje aplikacije u rad.

U sredu ćemo razmotriti vaše nalaze, proveriti ih dodatnim slučajevima i razgovarati o tome kako se donose odluke kada se pojave novi dokazi.

Za pripremu je dovoljno da možete da pročitate jednostavnu TypeScript funkciju, razumete osnovni test i pregledate izmene u kodu.

Nije potrebno da unapred pokrećete aplikaciju, koristite plaćeni model ili pripremate sopstveni projekat iz prethodne nedelje.

9. Pitanja za razmišljanje

Pre predavanja razmislite o sledećim pitanjima:

1. Ako AI aplikacija uspešno odgovori na deset pitanja, šta smo time zaista dokazali?

2. Ako aplikacija da pogrešan odgovor, kako biste utvrdili da li je problem u podacima, pretrazi, modelu ili aplikacionom kodu?

3. Ako Codex popravi problem i svi testovi prođu, koje dodatne informacije biste želeli pre nego što prihvatite tvrdnju da je problem rešen?

4. Ako nova verzija aplikacije postiže bolji rezultat evaluacije, ali koristi drugačije podatke i model, možemo li poboljšanje pripisati izmeni koda?

5. Da li biste pustili aplikaciju u produkciju ako prolazi 99% testova, ali preostali 1% uključuje ozbiljan bezbednosni propust?

Ne morate da pripremate pisane odgovore.

O ovim pitanjima razgovaraćemo tokom predavanja.

Glavna poruka za ovu nedelju

Dobar odgovor nije dovoljan dokaz da AI aplikacija pouzdano radi.

Za odgovornu isporuku softvera potrebno je da razumemo očekivano ponašanje, proveravamo rezultate, istražujemo greške i donosimo odluke na osnovu dokaza.

Pouzdanost ne pretpostavljamo — proveravamo je.